

Design and Implementation of a 2D Arcade Racing Game on the DE10-Standard FPGA Using Verilog HDL

1st Muhamad Afdhu

Department of Electrical Engineering
Telkom University
Bandung, Indonesia
cherryrebellion@student.telkomuniversity.ac.id

2nd Rangga Aryo Wicaksono

Department of Electrical Engineering
Telkom University
Bandung, Indonesia
ranggaaw@student.telkomuniversity.ac.id

Abstract— *Field-programmable gate arrays (FPGAs) have become a viable platform for implementing interactive digital systems that require deterministic timing and parallel hardware processing. This paper describes the design and implementation of a 2D arcade racing game on the DE10-Standard FPGA development board using Verilog HDL. The system is organized into modular hardware components comprising a VGA controller for video output, a PS/2 keyboard interface for user input, an audio module for sound generation, and game logic modules that manage object movement, collision detection, score calculation, and game-state transitions. The design was synthesized and deployed on the target FPGA platform, followed by functional verification of each module and the integrated system. Experimental results demonstrate that the implemented game executes reliably in real time, producing stable graphical output, responsive user interaction, and consistent gameplay behavior without software-based processing. The modular architecture improves design maintainability while providing a practical framework for extending game functionality. The implementation confirms that FPGA-based hardware design using Verilog HDL offers an effective approach for developing real-time interactive arcade applications with predictable system performance.*

Keywords— *Arcade game, DE10-Standard, FPGA, PS/2 keyboard, Verilog HDL, VGA controller*

I. INTRODUCTION

The development of digital systems is driving the use of Field Programmable Gate Arrays (FPGAs) as an implementation platform capable of providing parallel processing, low latency, and deterministic response times[1], [2]. In contrast to systems that rely on sequential execution of software, FPGAs implement logic functions directly on the hardware so that multiple processes can be executed simultaneously[2]. These characteristics make FPGAs suitable for applications that require high synchronization between input, logic processing, and visual output, including digital gaming systems that operate in real-time[1], [5].

The implementation of digital games on FPGAs has become one of the topics that has been widely developed as a medium for evaluating the integration of hardware modules. Several studies have succeeded in realizing the game of Pong using Verilog HDL by utilizing all game logic on FPGAs[1], [3], [4]. The results of such implementations show that graphics object processing, object position updates, collision detection, and display synchronization can be executed stably through hardware processing[3]. The findings show

that FPGAs have adequate capabilities to implement interactive gaming applications with consistent performance[4].

The successful implementation of digital games on FPGAs is influenced by the design of each module that makes up the system. The VGA controller generates horizontal and vertical synchronization signals while determining the coordinates of the pixels displayed on the screen[2], [5]. Verilog HDL provides flexibility in designing digital modules, such as display controllers, input interfaces, audio controllers, and game logic that can be developed and tested separately before being integrated into a single system[6], [7]. The modular approach simplifies the development process while increasing the ease of verification of each digital function[6].

Although the implementation of games on FPGAs has been widely reported, most studies still use a game of Pong or demonstration of simple graphical objects as the implementation medium[1], [3], [4], [8]. Racing-themed games have a higher complexity because they involve the simultaneous movement of multiple objects, continuous collision detection, score management, and synchronization between user input, graphical display, and audio output. This complexity presents its own challenges in the process of integrating digital modules into FPGAs.

This study implemented a two-dimensional arcade game with a racing theme on the DE10-Standard development board using Verilog HDL. The developed system integrates a VGA module, a PS/2 keyboard interface, an audio module, as well as game logic that regulates vehicle movement, enemy vehicles, collision detection, score calculation, and game state shifting. All functions are designed as interconnected hardware modules so that they can be executed directly on the FPGA.

This study contributes by implementing a two-dimensional racing game that integrates display, input, audio, and game logic modules into a single FPGA system using Verilog HDL. The implementation shows that FPGAs are capable of running interactive gaming applications in real-time by taking full advantage of the hardware processing on the DE10-Standard development board.

II. SYSTEM DESIGN

A. Overall System Architecture

Overall system architecture is designed by dividing the game into several interconnected functional modules. The module division includes input modules, game logic modules, VGA display generation modules, and audio modules. This approach simplifies the implementation process because each module has clear responsibilities and can be verified separately before full system integration is carried out.

Game starts on the Start Screen page waiting for commands from the user. Once the command is received, the system runs the countdown process as a pause before the game starts. During the game, the player's vehicle is controlled using the PS/2 keyboard to move to the left or right lane to avoid the opponent's vehicle moving from the top to the bottom of the screen. The score will increase each time the player manages to pass the obstacle, while the number of lives will decrease in the event of a collision. The game ends when all lives are exhausted and the system displays the Game Over page before the reset process is performed.

The relationship between the components of the system is shown in **Fig. 1**. The 50 MHz clock signal from the oscillator on the FPGA is first processed by the frequency divider module to generate the clock required by each module, including the VGA pixel clock. Input from the PS/2 keyboard as well as the buttons and switches on the board are received by the input interface module, then passed to the game logic module to set the vehicle position, enemy vehicle movement, score calculation, number of lives, and game status. The processed data is then sent to the VGA module to generate pixel coordinates and RGB color signals displayed on the monitor. At the same time, the game status is also sent towards the audio module so that sound effects can be generated according to the events that occur during the game.

The integration of all these modules allows the game system to be run entirely by FPGA hardware without the need for additional processing units. The separation of functions in each module also simplifies the process of developing, testing, and maintaining the system during implementation.

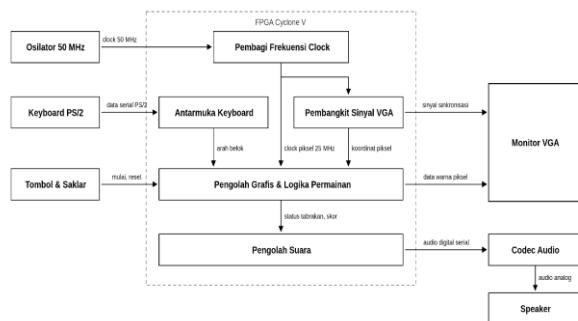


Fig. 1 Block Diagram of the 2D Arcade Racing Game System

B. Maintaining the Integrity of the Specifications

Implementation of the system was carried out using the DE10-Standard development board as a medium for the

realization of all digital modules designed with Verilog HDL. The FPGA on the board serves as a processing center that integrates game modules, VGA controllers, PS/2 keyboard interfaces, and audio modules into a single system. The synthesis and programming process of the device is carried out using Intel Quartus Prime before the system is tested directly on the hardware.

User interaction is done through the PS/2 keyboard as an input device, while the system output is displayed on the monitor using the VGA interface. In addition to producing a real-time visual display, the system also utilizes the audio codec available on the DE10-Standard to produce sound effects during the game. All hardware used in the implementation of this study is summarized in **Table I**.

TABLE I. HARDWARE REQUIREMENTS

No.	Hardware Device	Function
1	DE10-Standard FPGA Board	Implements the entire game system.
2	VGA Monitor	Displays the game graphics.
3	PS/2 Keyboard	Provides user input for vehicle control.
4	WM8731 Audio Codec	Generates audio output through the I ² S interface.
5	Computer/Laptop	Develops the Verilog HDL code, compiles the design, programs the FPGA, and performs system testing.

C. Functional Design

Functional design of the system follows the sequence of game operations starting from the initialization process to the end of the game, as shown in Fig. 2. After the FPGA finishes configuring, the system initializes the game parameters, which are the score value to 0 and the number of lives as many as three. Next, the home screen is displayed and the system waits for feedback from the user to start the game.

Once the button starts being pressed, the system runs a countdown process before the game starts. During the game, the control module continuously reads input from the PS/2 keyboard to determine the vehicle's movement to the left or right lane. The position of the player's vehicle and the enemy's vehicle is then updated on each update cycle, then each frame is sent to the VGA module to be displayed on the monitor. Once the position of all objects is updated, the system performs a collision detection process between the player's vehicle and the enemy vehicle. If there is no collision, the score will increase and the game continues by

updating the position of all objects in the next cycle. Conversely, when a collision is detected, the number of lives is reduced by one and the crash sound effect is activated as feedback to the user.

The system then evaluates the number of lives still available. The game will continue if there are still lives left, while the Game Over status is displayed when all lives have been depleted. At that stage the game is stopped and the system waits for a reset command from the user. Once the reset button is pressed, all game parameters are restored to their original state so that the game can be restarted.

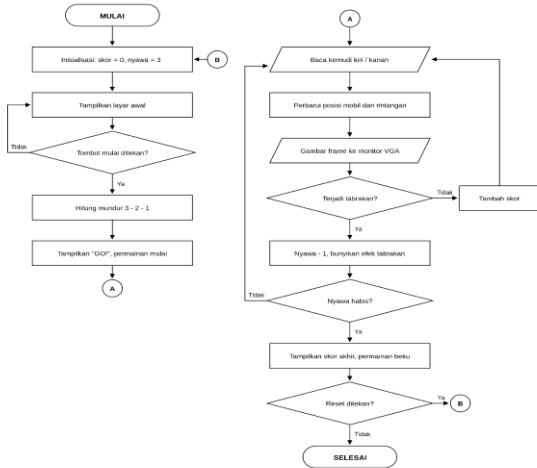


Fig. 2 Functional Flow of the 2D Arcade Racing Game

III. RESULTS AND DISCUSSION

A. Hardware Implementation and Module Integration

The game system was successfully implemented on the DE10-Standard development board using the Verilog Hardware Description Language (Verilog HDL). The implementation is carried out in a modular manner by dividing the system into several main components, namely the VGA controller, the PS/2 keyboard interface, the game logic, the rendering module, and the audio system. Each module is developed and tested separately before being fully integrated into a single game system that runs in real-time on FPGAs.

The implementation stage began with the development of a VGA controller as a basic module that produces pixel coordinates, horizontal synchronization (HSYNC), and vertical synchronization (VSYNC) signals. The resulting coordinates are used by the render_static.v module to determine the color of each pixel so that the entire game object can be displayed in the appropriate position. Furthermore, the PS/2 keyboard interface module is integrated to translate the scanned code into a control signal used by the game's logic. The logic module then regulates the player's vehicle movement, obstacle generation, collision detection, *score* updates, *lives management*, and game status shifts. In the final stage, the audio system is connected with the WM8731 codec via the I²S interface so that sound effects can be played according to the game conditions.



Fig. 3 Hardware Implementation of the Proposed Racing Game

B. Core Verilog Implementation

The proposed arcade racing game was implemented using several Verilog HDL modules, where each module performs a dedicated function within the overall system. The implementation adopts a modular design to simplify development, debugging, and hardware integration. The primary modules include the VGA timing generator, player movement controller, collision detection unit, score and lives management, rendering engine, and audio controller. The following subsections highlight the core implementation of the proposed system.

1) VGA Timing Generation

VGA controller generates the pixel coordinates and synchronization signals required to display graphics on the monitor. The implementation adopts the standard VGA resolution of 640×480 pixels at 60 Hz, which is fully supported by the DE10-Standard FPGA board. Two synchronous counters are used to generate the horizontal and vertical pixel coordinates. The horizontal counter increments on every pixel clock cycle, while the vertical counter advances after one horizontal scan line has been completed. These coordinates are subsequently used by the rendering module to determine the color of each displayed pixel.

Listing 1. VGA Timing Generation

```
always @(posedge pixel_clk) begin
    if (reset) begin
        x <= 0;
        y <= 0;
    end
    else if (x == H_TOTAL-1) begin
        x <= 0;
        y <= (y == V_TOTAL-1) ? 0 : y + 1;
    end
    else
        x <= x + 1;
    end
    assign video_on = (x < H_VISIBLE) && (y < V_VISIBLE);
    assign hsync = ~(x >= H_SYNC_START && x < H_SYNC_END);
    assign vsync = ~(y >= V_SYNC_START && y < V_SYNC_END);
```

Listing 1. illustrates the timing generation mechanism implemented in the VGA controller. The horizontal counter is incremented on every pixel clock cycle. Once the end of a scan line is reached, the horizontal counter is reset and the vertical counter is incremented. The generated pixel coordinates are subsequently used by the rendering module to determine the location and color of every object displayed

on the monitor. This implementation ensures stable synchronization between the FPGA and the VGA display throughout the gameplay.

2) Player Movement Controller

Player vehicle is controlled through the PS/2 keyboard interface. Keyboard scan codes are translated into movement commands, allowing the vehicle to move left or right by updating the car_offset parameter.

Listing 2. Player Movement Logic

```
always @(posedge clk) begin
    if (reset)
        car_offset <= 11'sd0;
    else if (game_started && !game_over) begin
        if (steer_left && !steer_right)
            car_offset <= (car_offset >
car_offset_min_final) ?
car_offset - CAR_STEP :
car_offset_min_final;
        else if (steer_right && !steer_left)
            car_offset <= (car_offset <
car_offset_max_final) ?
car_offset + CAR_STEP :
car_offset_max_final;
    end
end
```

The movement controller continuously monitors the steering commands received from the keyboard interface. Before updating the vehicle position, the logic verifies the predefined left and right boundaries of the road to prevent the vehicle from leaving the playable area. This boundary-checking mechanism improves gameplay stability while maintaining responsive user control during real-time operation.

3) Bounding Box Collision Detection

Collision detection is implemented using the Bounding Box Collision Detection method by comparing the boundaries of the player vehicle and the obstacle.

Listing 3. Bounding Box Collision Detection

```
wire obstacle_car_overlap =
    obstacle_active &&
    (obs_left_s < car_right_s) &&
    (obs_right_s > car_left_s) &&
    (obs_top_s < car_bottom_s) &&
    (obs_bottom_s > car_top_s);

wire collision_edge =
    obstacle_car_overlap &&
    !obstacle_overlap_prev;
```

The collision detection logic evaluates the overlap between the bounding boxes of the player vehicle and the obstacle. A collision is declared only when all horizontal and vertical overlap conditions are satisfied simultaneously. Furthermore, the collision_edge signal is employed to detect only the first occurrence of a collision event, preventing repeated collision processing while the two objects remain in contact across multiple clock cycles. This approach provides a simple and computationally efficient solution for real-time FPGA implementation.

4) Score and Lives Management

Score management module updates the player's score according to the game state, while the lives management

module determines whether the game should continue or enter the Game Over state.

```
always @(posedge clk) begin
    if (obstacle_reached_end) begin
        score <= score + POINTS_PER_DODGE +
            combo_bonus +
            (near_miss_latch ?
NEAR_MISS_BONUS : 16'd0);
        combo_count <= combo_count + 8'd1;
    end

    if (collision_edge) begin
        combo_count <= 8'd0;
        if (lives > 3'd1)
            lives <= lives - 3'd1;
        else
            game_over <= 1'b1;
    end
end
```

Implementation increases the player's score whenever an obstacle is successfully avoided. Additional rewards are provided through the combo bonus and near miss bonus mechanisms, encouraging continuous successful gameplay. Conversely, when a collision is detected, the system resets the combo counter, decreases the number of remaining lives, and applies a score penalty. Once all lives have been exhausted, the game_over signal is asserted, causing the system to terminate the gameplay and display the Game Over screen. This implementation integrates scoring, player status, and game-state transitions within a single control mechanism, ensuring consistent gameplay behavior throughout the system.

C. Functional Testing Results

Functional evaluation was conducted after all hardware modules had been successfully integrated on the DE10-Standard FPGA board. The objective of the testing was to verify that each module operated according to the proposed design under real-time conditions. The evaluation covered the complete gameplay sequence, including the initial interface, gameplay execution, game termination, and the functionality of all integrated hardware modules.

1) Game Interface Verification

The proposed arcade racing game was evaluated by observing each gameplay state displayed through the VGA interface. **Fig 4.** illustrates the Start Screen that appears immediately after system initialization. At this stage, the game remains in the idle state while waiting for the player to start the game through the PS/2 keyboard. All game variables, including the player position, score, lives, and obstacle status, are initialized before the gameplay begins.



Fig. 4 Start Screen of the Proposed Arcade Racing Game

After the Start command is received, the system automatically enters the countdown stage shown in **Fig. 5**. During this phase, the gameplay logic remains inactive while a countdown sequence is displayed to provide a short preparation period before the game starts. Once the countdown reaches its final state, the system activates all gameplay modules simultaneously.



Fig. 5 Countdown Screen Before Gameplay

Fig 6. presents the main gameplay interface after the countdown has finished. The VGA controller continuously generates pixel coordinates, while the rendering module updates the display according to the current game state. The player vehicle responds to steering commands received from the PS/2 keyboard, obstacles move continuously toward the bottom of the screen, and the score and remaining lives are updated in real time throughout the gameplay.



Fig. 6 Main Gameplay Interface

2) Functional Testing

The functionality of each hardware module was verified after the complete system had been implemented on the DE10-Standard FPGA board. The testing focused on validating the VGA display, PS/2 keyboard interface, vehicle movement, obstacle generation, collision detection, score management, lives management, audio output, and overall hardware integration.

TABLE II. FUNCTIONAL TESTING RESULTS

No.	Tested Function	Expected Result	Result
1	VGA display	Game interface is correctly displayed on the VGA monitor	Passed
2	PS/2 keyboard interface	Player vehicle responds correctly to keyboard input	Passed
3	Player movement	Vehicle moves according to user commands	Passed

No.	Tested Function	Expected Result	Result
4	Obstacle movement	Obstacles appear and move continuously	Passed
5	Collision detection	Collision between the player and obstacles is correctly detected	Passed
6	Score management	Score increases after successfully avoiding obstacles	Passed
7	Lives management	Lives decrease after collision events	Passed
8	Audio system	Sound effects are generated according to game events	Passed
9	System integration	All hardware modules operate simultaneously without errors	Passed

When all player lives have been exhausted, the system enters the Game Over state as illustrated in **Fig. 7**. At this stage, vehicle movement, obstacle generation, score updates, and other gameplay processes are terminated, while the Game Over screen remains displayed until the player starts a new game.

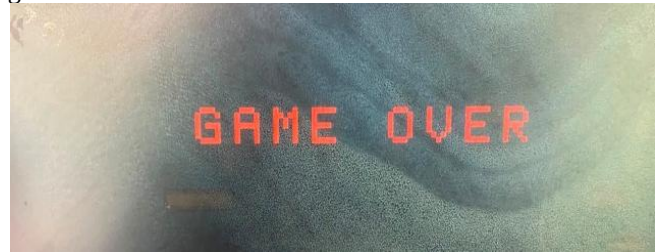


Fig. 7 Game Over Screen

Functional testing verified that all implemented modules operated as intended. Stable VGA synchronization ensured correct visualization of the game interface, while the PS/2 keyboard interface provided accurate and responsive player control. Obstacle generation, collision detection, score management, and lives management functioned consistently throughout gameplay. Sound effects were also successfully produced through the WM8731 codec using the I²S interface. These results confirm that the proposed arcade racing game was successfully implemented and operates reliably in real time on the DE10-Standard FPGA platform.

IV. CONCLUSION

A real-time 2D arcade racing game has been successfully implemented on the DE10-Standard FPGA using Verilog HDL. Experimental results verified that the VGA controller, PS/2 keyboard interface, rendering engine,

game logic, collision detection, score and lives management, and audio subsystem operated correctly after hardware integration. The implemented system produced stable graphics, responsive user control, reliable gameplay mechanics, and synchronized audio output throughout the testing process. These results demonstrate that FPGA-based implementation is suitable for developing real-time arcade games using a modular hardware architecture. Future work can extend the system by adding multiple difficulty levels, improved graphics, and multiplayer functionality.

REFERENCES

- [1] T. Saha, A. K. T. Ahmed, and S. M. S. Alam, "Design and Implementation of Multi Player Pong Game using Altera DE2 Board," *International Journal of Intelligent Systems and Applications*, vol. 15, no. 6, pp. 25–37, 2023, doi: 10.5815/IJISA.2023.06.03.
- [2] A. Chinchankar, P. H. Chandankhede, and A. Titarmare, "VGA Controller Design & Implementation on FPGA," in *2023 IEEE Global Conference for Advancement in Technology (GCAT)*, Bengaluru, India, 2023, doi: 10.1109/GCAT59970.2023.10353298.
- [3] T. R. Padda, K. K. Reddy, S. Sreedhari, and V. Gayathri, "Hardware Design of a Real-Time Pong Game on FPGA with VGA Display Support," *International Journal of Engineering Research & Technology (IJERT)*, vol. 14, no. 6, pp. 337–343, 2025.
- [4] K. L. Manogna, P. Swathi, C. Poornima, B. R. Onteddu, and R. Siddaiah, "Real-Time Implementation of a Pong Game on FPGA Using Verilog HDL and VGA Display Interface," *International Journal of Innovative Research in Technology (IJIRT)*, vol. 13, no. 1, pp. 4745–4751, 2026.
- [5] Z. T. He, "Color Image VGA Display Based on FPGA and OV7725," *Open Access Library Journal*, vol. 10, pp. 1–10, 2023, doi: 10.4236/oalib.1110556.
- [6] Z. Yan, "The Application of Verilog in the Development of Casual Games," *Applied and Computational Engineering*, vol. 76, pp. 245–249, 2024, doi: 10.54254/2755-2721/76/20240600.
- [7] X. Wang, "FPGA-Based Sokuban-Like Fun Logic Learning Program with Combinatorial Logic Blocks," *Applied and Computational Engineering*, vol. 72, pp. 210–216, 2024, doi: 10.54254/2755-2721/72/20241022.